SMT × MemUP: текст к презентации 26.10.2026

Что говорить на каждом слайде и короткие ответы на ожидаемые вопросы. Числа — как на слайдах (состояние 26.10.2026, 07:30). Источники чисел — deck/SOURCES_AUDIT_RU.md; код — SMT_MemUP_EXPLAINABLE.ipynb.

1. SMT × MemUP: Stack и MQAR

Сегодня две задачи по порядку: сначала Stack, потом MQAR. Для каждой — данные и пример, окна, схема, loss, модели и только потом результаты. Новые запуски на RTX 3090 (seeds 201–203, независимый набор) и старые архивные (seed 107 или 7/17/27) нигде не смешаны в одном сравнении.

Главное: на Stack итоговая RNN после SMT → DMT даёт 95,5% на трёх seeds, полный BPTT той же RNN — 99,6%, BPTT-1 — 31,2% (пока один seed). На коротком MQAR (T = 16, N = 2) teacher SMT выучивает связывание ключа и значения (99,1%). Вывод прошлой презентации, что восемь токенов памяти этого не могут, я снимаю. Итоговая RNN на MQAR пока заметно ниже teacher.

— **Почему цифры теперь другие, чем 20.10?**
Новый протокол: три seeds, независимый набор, длиннее бюджеты, постоянный LR. Старые результаты остались в архиве и подписаны как старые.

2. Stack: вход — ID операции, ответ — снятый со стека элемент

Вход — это не текст, а готовые целые ID: 0 — pop, 1..4 — push этого элемента. Ответ стоит на той же позиции: на pop — снятый элемент, на push — −1. −1 означает «ответа нет», такие позиции маскируются; это не шестой класс.

Пример из нашего обсуждения: x = [1, 2, 0, 3, 0, 0], y = [−1, −1, 2, −1, 3, 1]. Стек после каждого шага, позиция push-источника и delay — в таблице. delay = позиция pop − позиция push, это расстояние в данных, а не смещение j декодера.

У teacher и у RNN разные обучаемые таблицы эмбеддингов, обе размером 5 × 64.

— **Откуда «delay ≥ 9»?**
Между push и его pop лежит сбалансированный кусок, поэтому delay в Stack всегда нечётный: порог 8 фактически означает 9.

— **Что значит «дальний ответ» в метриках?**
Pop с delay ≥ 9; на них считается основная accuracy.

3. Окна SMT на этом примере

Для каждого якоря t строятся три окна: x_ctx — входы до t включительно (слева паддинг), x_fut — входы после t, y_fut — ответы начиная с t. Выход j декодера читает память m_t и x_{t+1..t+j} и предсказывает y_{t+j}.

Якорь t = 2: j = 0 предсказывает y_2 = 2 по одной памяти; j = 2 — y_4 = 3, но его push (t = 3) уже виден во входах декодера; j = 3 — y_5 = 1, его push был на t = 0, значит ответ есть только в памяти. Якорь t = 3: j = 1 и j = 2 — оба ответа только в памяти.

T_f = 2 здесь только для иллюстрации границы ближних и дальних целей; в экспериментах T_f = 8. В реальной длине T = 64 у якоря t = 10 — 11 настоящих входов и 53 паддинга. Все эти выдержки печатает настоящая функция windows(), а assert проверяет, что выход j не видит x_fut[j:].

– **Почему нет сдвига на один токен, как в LM?**
  Ответ относится к текущему входу (pop), а не к следующему токену; память m_t уже прочитала x_t, поэтому j = 0 отвечает на y_t.

4. Схема обучения и инференса

SMT: энкодер teacher читает x_ctx вместе с восемью обучаемыми регистрами; выходы на позициях регистров после RMS-нормы — это память m_t размером 8 × 64. Декодер teacher читает m_t и будущие входы. Updater RNN f получает m_{t−1} энкодера и x_t и учится повторить m_t.

DMT: энкодер заморожен и даёт только цели. RNN разворачивает своё состояние $\hat{m}$ и учится повторять m_t; detach режет градиент между шагами, но не обнуляет состояние.

Инференс: только f и readout RNN. Нет энкодера, нет декодера teacher, нет правильных ответов и нет отбора дальних целей.

– **Регистры и память — одно и то же?**
  Нет: регистры — параметры, одинаковые для всех входов; память — выход энкодера на их позициях, зависит от x_0..x_t. В ноутбуке это проверено assert.

5. Функция потерь

L_SMT = c_dec (L_near + α L_far) + c_dyn L_dyn + c_unif L_unif с весами 1; 0,1; 0,001 и α = 1. L_near — CE по смещениям j < 8, L_far — CE по двум выбранным дальним целям (если правило far = none, этот член ноль). L_dyn — MSE между f(m_{t−1}, x_t) и m_t. L_DMT = 10 · MSE + CE, где CE считается по отцепленным состояниям и учит только readout.

Функции в ноутбуке совпадают с кодом, которым обучены все новые запуски, бит в бит: 21 сравнение, расхождение 0.

– **Кто получает градиент от L_dyn?**
  Updater f и энкодер (через m_{t−1} и m_t); в абляциях цель или весь L_dyn отцепляются.

6. Stack: модели

Базовый вариант — SMT с коротким окном без дальних целей. Варианты меняют одну вещь: правило выбора двух дальних целей (по ошибке детектора без памяти, по локальной неожиданности, случайно), L_dyn, длину окна. Полный BPTT и BPTT-1 — та же RNN, обученная с нуля без teacher. «Диагностика: источник уже в памяти» использует разметку генератора, у метода такой информации нет, поэтому это не метод, а верхний ориентир.

– **Почему разные бюджеты у старых строк?**
  Это старые протоколы; они показаны отдельно и в сравнение с новыми не идут.

7. Stack, новые запуски

Teacher выходит на ~100% к шагу ~4000–6000. Итоговая RNN после DMT: 95,0 / 94,8 / 96,8%. Полный BPTT той же RNN: 99,3 / 99,9 / 99,5%. BPTT-1: 31,2% на seed 201, остальные считаются. Дальние цели по ошибке без памяти: 92,4 / 95,1% (seed 203 в очереди), то есть не выше базы.

У seed 203 история SMT сохранилась только до шага 6500 — сбой синхронизации; итоговые числа это не затрагивает, кривая просто не дорисована.

– **Почему SMT → DMT ниже полного BPTT?**
  Наблюдение на этом бюджете; причина не изолирована. Это направление для следующего опыта, а не вывод.

8. Старый протокол: окно, T_f, компоненты

Три панели — три разных протокола, между собой их не сравниваю. Короткое окно лучше полного при малом бюджете (57,4 против 49,1%, всё недообучено). На одном seed: при T_f = 1 RNN 37,9%, при 8 — 94,5%; без L_dyn — 66,0%. Teacher при этом почти 100% во всех вариантах: хороший teacher не гарантирует пригодную для RNN динамику.

Новые запуски seed 201 повторяют главное: без L_dyn 65,5%. Отцепление цели L_dyn дало 98,5% против 95,0% у базы — это один seed.

9. Дальние цели: три метрики

Teacher, probe памяти и итоговая RNN — разные метрики. Probe — замороженный декодер teacher на дальних парах, чей push уже в памяти. С дальними целями probe растёт (34% → 47–62%, у диагностики 73%), итоговая RNN — нет (52–57%). При длинном обучении (v4, seed 107) probe 99% против 47%, а RNN 98,6 против 98,1%.

Вывод осторожный: дальние цели меняют память teacher; устойчивого выигрыша итоговой RNN пока нет. Я не утверждаю, что добавка бесполезна всегда — сравнение «локальная неожиданность против случайных» в новом протоколе ещё считается.

– **Почему probe растёт, а RNN нет?**
  Не установлено. Варианты: Stack насыщается; DMT не переносит дополнительную информацию; она доступна teacher, но не нужна RNN. Текущие контроли их не разделяют.

10. MQAR: задача и пример

Генератор Zoology. Одна пара — два ID: ключ на позиции 2i, значение на 2i+1; ключи из 1..63, значения из 64..127. Потом каждый ключ запрашивается ровно один раз, запрос — это сам ID ключа, метка — значение — стоит на позиции запроса. Пример T = 16, N = 2: пары (54 → 76) и (8 → 96); на t = 4 запрос 8 → 96, на t = 8 запрос 54 → 76.

Accuracy по всему словарю (128 классов) — основная. Accuracy среди записанных в последовательности значений — диагностика, у неё угадывание 1/N.

11. MQAR: схема и режимы чтения

Обучение то же, что в Stack, но окно покрывает весь пример (T_f = T) и дальних целей нет. Режимы оценки teacher: j = 0 на позиции запроса (энкодер видел запрос); память до запросов (энкодер видел только пары, ключ запроса приходит через декодер); диагностика, где декодер читает все выходы энкодера, а не восемь токенов.

12. MQAR: loss и модели

Модели: SMT → DMT; teacher только с L_near (без RNN) — чтобы отделить вклад L_dyn; диагностика со всеми выходами энкодера; та же RNN с полным BPTT; causal Transformer 2 слоя d = 64. Бюджеты в таблице.

13. MQAR T = 16: кривые

Teacher стоит на плато 50% — это угадывание одного из двух значений — и между шагами 5000 и 6000 прыгает к ~100%. Seed 203 прыгнул на самой последней точке. Декодер со всеми выходами энкодера прыгает около шага 1500: доступ ко всем выходам ускоряет обучение, но восемь токенов тоже справляются.

Полный BPTT: за 6000 шагов не сошёлся. С бюджетом до 60 000 seed 202 поднимается ступенями и доходит до 99% к шагу ~51 000, seed 201 за 60 000 — 56%, seed 203 ещё идёт.

- **Почему 20.10 было 50,8%?**
  Тот же бюджет 6000 шагов, но LR по косинусу падал до 3e-5, а скачок происходит около 5000–6000 шагов. Плюс другой seed и tuning-набор. Причиной было расписание, а не ёмкость памяти.

14. MQAR T = 16: независимый набор

Transformer 98,7%; teacher j = 0 99,1%; память до запросов 99,9%; teacher только с L_near 99,5%; со всеми выходами энкодера 100%. Итоговая RNN до DMT 44,7%, после DMT 57,6%; полный BPTT 6000 шагов 45,2%, до 60 000 — 55,7 / 99,3%.

Наблюдение: итоговая RNN после DMT заметно ниже teacher. Причину не называю: DMT всего 2000 шагов, а та же RNN с полным BPTT задачу решает, но не на каждом seed.

15. MQAR T = 32 и старый пилот T = 64

T = 32, N = 4: teacher выучил задачу на одном seed из трёх (95,9%), частично на втором (74,9%), на третьем остался на угадывании (24,8%) за 6000 шагов. Итоговая RNN 58,2 / 27,4 / 24,7%. Transformer 99,3–99,7%. Полный BPTT в очереди.

Старый пилот T = 64, N = 8 — другой протокол (seed 107, косинусный LR). Повторять его на новом протоколе нужно до любых выводов о T = 64.

16. Итоги

Установлено: на Stack итоговая RNN 95,5%, полный BPTT 99,6%, BPTT-1 около 31%; качество RNN зависит от L_dyn и T_f; дальние цели улучшают память teacher, но не итоговую RNN. На MQAR T = 16 teacher учится связыванию, RNN — ниже.

Исправлено: «восемь токенов не хранят пары» — неверно; «все выходы энкодера: 12,8 → 100%» на T = 16 — это разница в скорости; «полный BPTT на уровне угадывания» — 6000 шагов было мало.

Считается: BPTT-1 seeds 202–203, «L_dyn полностью отцеплен», «локальная неожиданность» против случайных, полный BPTT на MQAR seed 203 и T = 32.

– **Что дальше?**
  Длиннее DMT на MQAR и полный BPTT на T = 32; повтор T = 64 на новом протоколе; сравнение правил отбора дальних целей на трёх seeds.

Карта: слайд → данные → ноутбук

слайд	данные	ноутбук SMT_MemUP_EXPLAINABLE
2	пример из обсуждения	§2.1 (таблица + assert)
3	функция windows()	§3, §3 t = 10
4, 11	схема по коду smt_loss / dmt_loss / инференс	§5, §8
5, 12	smt_loss, dmt_loss, bptt_loss	§4 (формулы), §7, §11 (сверка)
7, B	results_3090/notebook_runs/stack_*/{smt,dmt,bptt}_history.csv, summary.json	§10
8	archive/outputs_v2_window/table_validation_by_seed.csv, archive/outputs_ablation/table_ablation.csv	§10 (таблица абляций)
9, C	archive/outputs_v2_window/…; archive/runs/stackv4_d459c6921638/diag/*	§6 (выбор дальних целей)
10	генератор Zoology, seed derive_seed(901, 0) = 4155743513, батч из 1 последовательности	§2.2, §2.3, §8.1
13, 14	results_3090/notebook_runs/mqar_T16N2_*	§9 (метрики), §10
15, D	results_3090/notebook_runs/mqar_T32N4_*; archive/runs/jobs/bneck_*_T64.log; archive/outputs_mqar/table_mqarA_summary.csv	§10